QUIZLER: A DYNAMIC API-DRIVEN TRIVIA APPLICATION

PROJECT REPORT :

Submitted by:

Detail	Information
Student Name	Surya Prakash Sharma
SAP ID	590027399
Program	Btech-CSE
Batch No	53

Table of Contents :

1. Title Page 1
2. Introduction 3
3. Problem Statement 4
4. Objectives 5
5. System Requirements 6
6. Implementation: Data Retrieval and Modeling 7
7. Implementation: Game Logic and Orchestration 8
8. Implementation: Graphical User Interface (GUI) 9
9. Output Screenshots 10

2. Introduction: Quizler - A Dynamic GUI Trivia Application :

This project introduces "Quizler," an interactive and dynamic Graphical User Interface (GUI) trivia application designed to overcome the limitations of traditional, command-line-based educational scripts. While functional, these older scripts lack user engagement and visual appeal. Furthermore, they typically rely on hardcoded data, which severely restricts both replayability and practical utility.

Quizler solves these problems by providing a visually engaging interface and ensuring a fresh set of trivia data is fetched from the internet every time the application runs. The architecture is built upon Object-Oriented Programming (OOP) principles, ensuring a clean separation between the data model, the graphical view, and the core game logic.

The application's core functionality involves connecting to the Open Trivia Database (OpenTDB), parsing the incoming JSON data, structuring this data into custom Python objects, and presenting the quiz to the user via a Tkinter-based dashboard. This approach mirrors real-world software development practices, where the front-end interface is completely decoupled from the back-end data gathering and business logic.

3. Problem Statement :

This project addresses two core limitations of basic programming projects, particularly in the context of data-driven applications like trivia games: **static, unengaging data presentation** and **inefficient manual data management**

Current Challenges:

1. **Tedious and Error-Prone Data Handling:** Manually creating and maintaining a large dataset, such as hundreds of trivia questions, by hardcoding it into a script is slow, highly susceptible to syntax and formatting errors, and requires constant developer effort for updates to keep the content fresh.
2. **Poor User Experience:** Running command-line Python scripts offers a non-intuitive and inaccessible experience for the average user. Modern consumer software must be visual, responsive, and dynamic, which a standard terminal cannot provide.

Project Solution:

The project overcomes these challenges by **automating the entire data-gathering process** and providing an accessible user interface. It communicates with a live web API to automatically source unique questions upon every execution, eliminating the need for a manually written or updated question database. This automated data stream is then encapsulated within a user-friendly graphical window, effectively transforming raw web data into accessible, dynamic consumer software.

4. Project Objectives :

The core objectives of this project are to build an automated, efficient system for dynamic data retrieval and processing.

- **Automation of Input:** Eliminate manual effort by removing the need for hardcoding or manually typing questions into the source code.
- **Data Acquisition:** Successfully fetch and decode JSON payloads from the OpenTDB web API.
- **User Interface Development:** Design an interactive Graphical User Interface (GUI) using the Tkinter library to provide immediate, color-coded visual feedback.
- **Architectural Design:** Strictly adhere to Object-Oriented Programming (OOP) principles, implementing distinct classes for the Data Model, Game Logic (Controller), and User Interface (View).

5. System RequirementsHardware Specifications :

Component	Minimum Requirement	Note
Processor	Intel i3 or equivalent	
RAM	4GB	
Network	Active Internet Connection	Essential for the `requests` module to fetch API data.

Software Specifications

Category	Item	Purpose

Language	Python (Version 3.14)	Core development language.
IDE	VS Code / PyCharm / Jupyter Notebook	Recommended Integrated Development Environments.
Built-in Libraries	`tkinter`	For developing the graphical user interface.
	`html`	For unescaping web-formatted text from the API.
Third-Party Library	`requests`	For executing HTTP GET requests to the OpenTDB server.

6. Implementation Details :

The project is developed using a modular, Object-Oriented Programming (OOP) approach, with its architecture distributed across distinct Python files.

A. Data Retrieval (`data.py`) :

This module is responsible for fetching questions from the Open Trivia Database. It uses the `requests` library to execute a `GET` request, specifically tailored to retrieve 10 boolean (True/False) questions by passing the necessary parameters. Critical to this process is the use of `response.raise_for_status()`, which ensures any HTTP errors are caught before the JSON payload is processed.

Code Example:
import requests

```
parameters = {"amount": 10, "type": "boolean"}

response = requests.get("https://opentdb.com/api.php",
params=parameters)
response.raise_for_status()
question_data = response.json()["results"]
```

B. Data Modeling (`question_model.py`) :

To maintain consistency and structure within the application, a `Question` class is implemented. Instead of passing around raw data dictionaries, the fetched data is converted into structured `Question` objects, each possessing dedicated `text` and `answer` attributes.

Code Example:

```
class Question:
    def __init__(self, q_text, q_answer):
        self.text = q_text
        self.answer = q_answer
```

The core logic of the quiz application is divided into two primary components: the **Game Logic** (`quiz_brain.py`) and the **Application Orchestration** (`main.py`).**Quiz Engine (`quiz_brain.py`)**

The `QuizBrain` class serves as the application's engine, managing the state and evaluation.

- **Function:** It accepts a list of `Question` objects, tracks the `question_number` and `score`, and processes user input.
- **Data Handling:** It employs `html.unescape()` to clean up web-sourced text before evaluation.
- **Evaluation:** The `check_answer` method compares the user's answer (case-insensitively) against the correct answer of the current question. It updates the score and provides a boolean result back to the user interface.

Code Excerpt for `QuizBrain.check_answer`:

```
import html

class QuizBrain:

    # ... __init__ and other methods ...

    def check_answer(self, user_answer):

        correct_answer = self.current_question.answer

        if user_answer.lower() == correct_answer.lower():

            self.score += 1

            return True

        else:

            return False
```

Application Setup (`main.py`) :

This file acts as the entry point, responsible for data ingestion and initialization.

- **Process:** It iterates through the raw API data (`question_data`), extracts the question text and correct answer, and uses the `Question` model to instantiate new objects.
- **Initialization:** These new `Question` objects are collected into the `question_bank` list, which is then used to initialize the rest of the application.

Code Excerpt for Setup Loop:

```
question_bank = []
for question in question_data:
    question_text = question["question"]
    question_answer = question["correct_answer"]
    new_question = Question(question_text, question_answer
question_bank.append(new_question)
```

E. User Interface (`ui.py`) :

The `QuizInterface` class, built with the `tkinter` library, manages the application's graphical components. It is responsible for:

1. **Initialization:** Setting up the main window and a `Canvas` area for displaying text.
2. **Interactivity:** Mapping class methods to the various interactive buttons.
3. **Visual Feedback:** Utilizing the `window.after()` method to create non-blocking delays of 1000 milliseconds, which provides visual feedback to the user before the quiz state transitions.

The `give_feedback` method demonstrates this visual feedback mechanism:

- It changes the `Canvas` background to green (`#90EE90`) for a correct answer or red (`#ee2400`) for an incorrect answer.
- It then calls `self.window.after(1000, self.get_next_question)` to pause for one second before loading the subsequent question.

-----**Code Snippet:**

```
from tkinter import *
from quiz_brain import QuizBrain

class QuizInterface:
    def __init__(self, quiz_brain: QuizBrain):
        self.quiz = quiz_brain
        self.window = Tk()
        self.window.title("Quizler")
        # Canvas and Button configurations...

    def give_feedback(self, is_right):
        if is_right:
            self.canvas.config(bg="#90EE90") # Green
        else:
```

```
        self.canvas.config(bg="#ee2400") # Red
    self.window.after(1000, self.get_next_question)
```

7. Application Screenshots :

Please include four large screenshots of the running application below, scaled to fill this entire page. These should demonstrate key stages of the program's operation and interface.

Figure 1: Initial Interface

Figure 2: Correct Answer Display

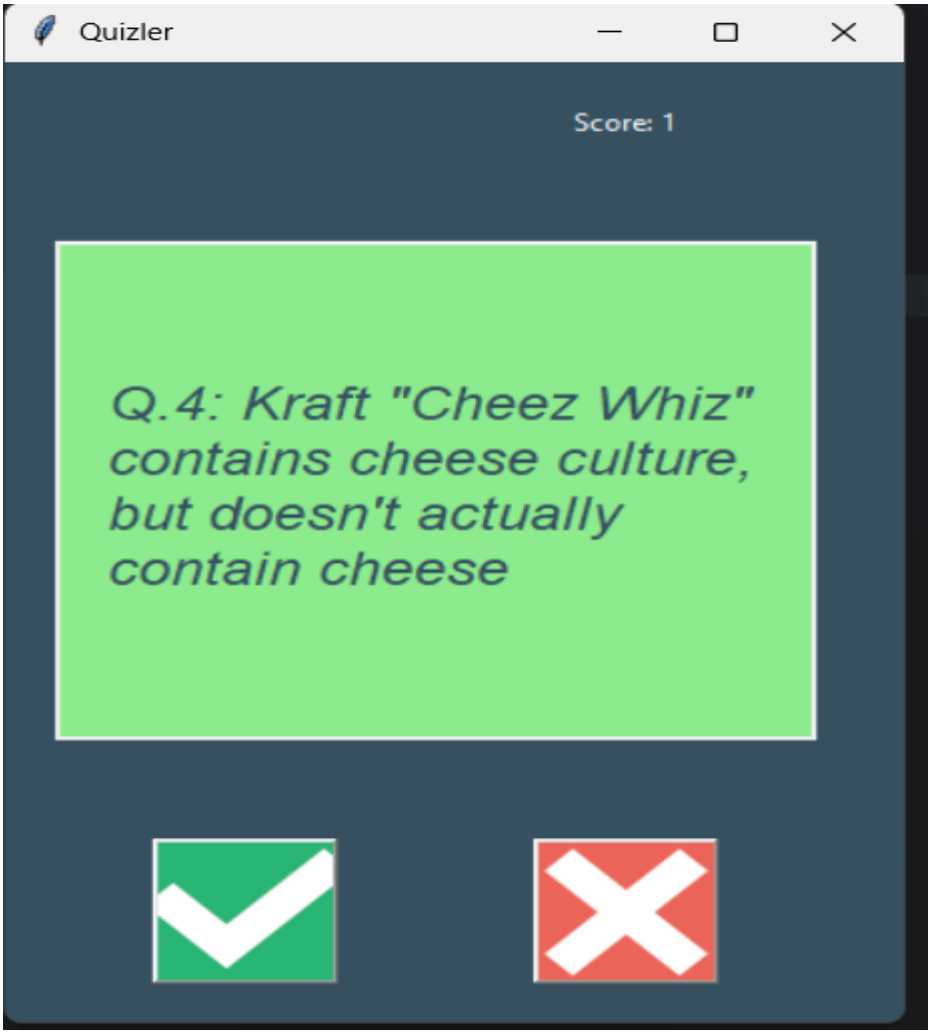


Figure 3: Incorrect Answer Display

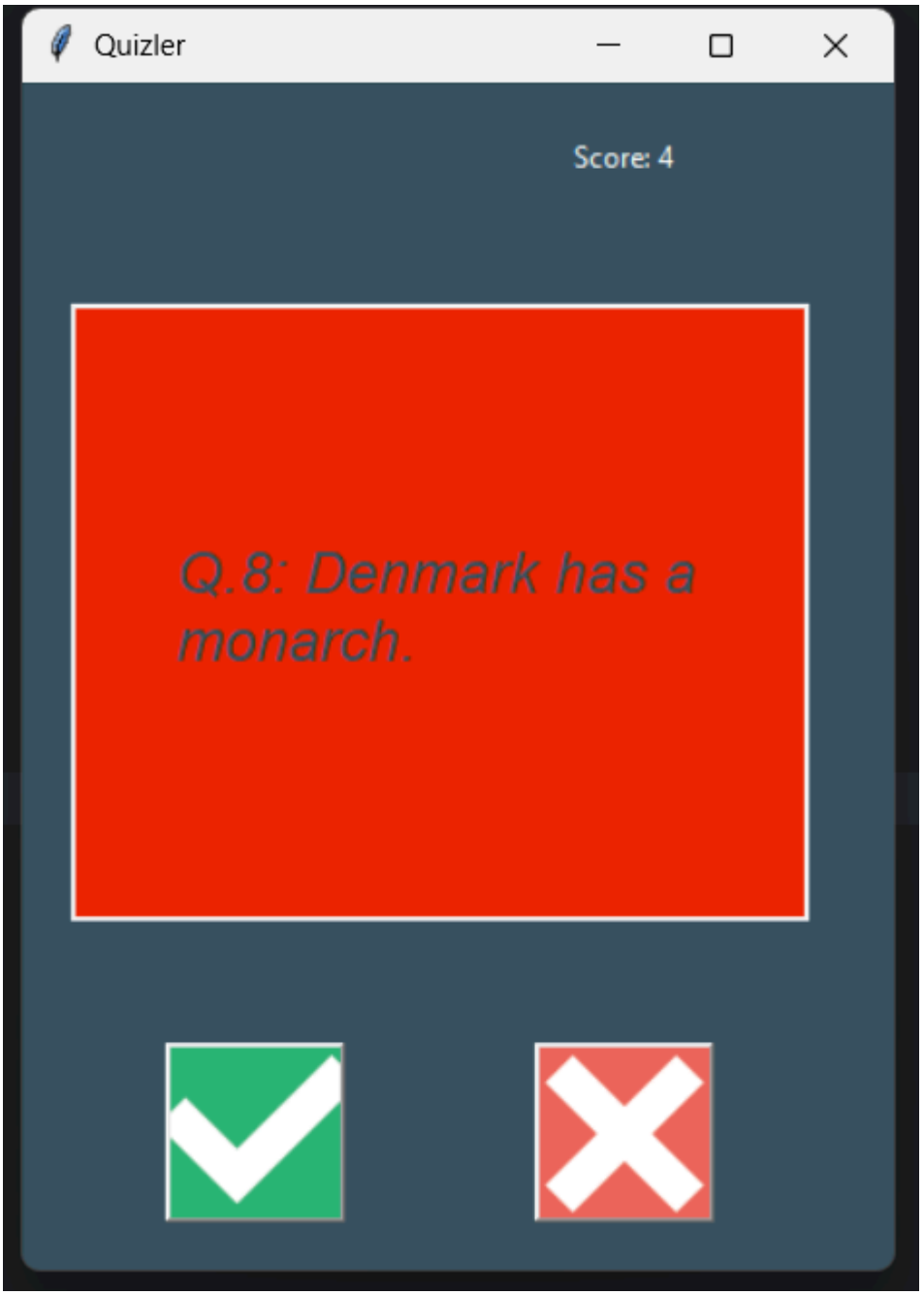
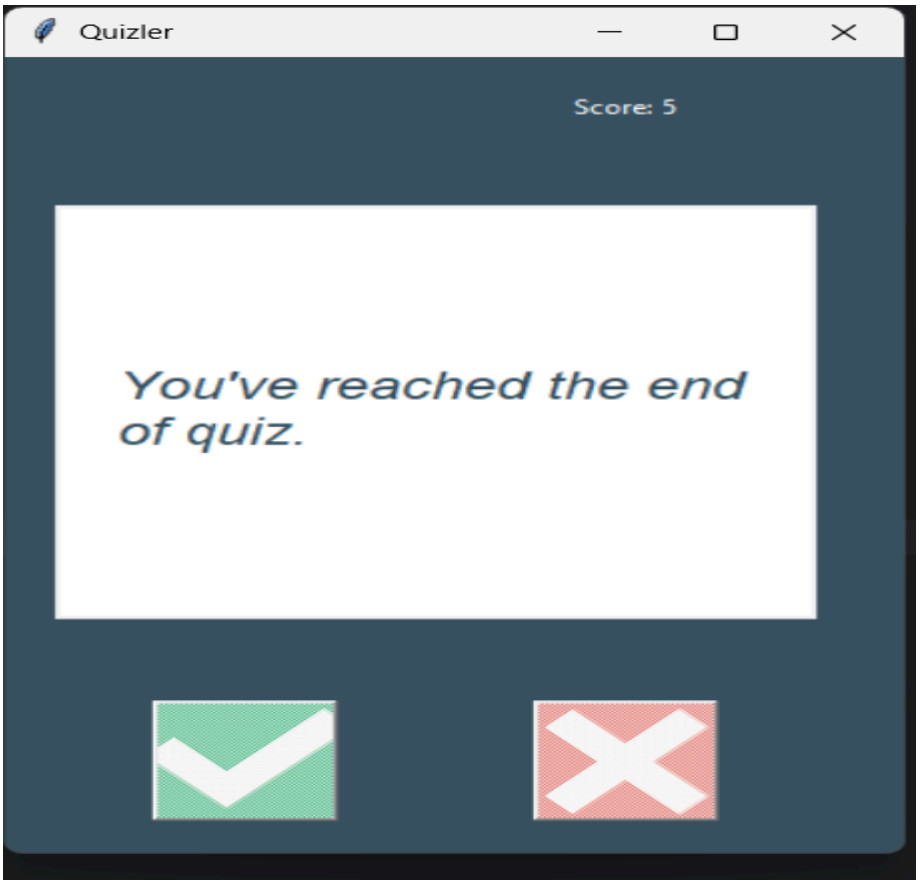


Figure 4: Final State



8. Conclusion :

This project successfully delivered a fully functional and highly interactive trivia application. A key achievement was the integration of the OpenTDB API, which automates the question generation process, thereby ensuring a unique and dynamic user experience with every program run.

All primary project objectives were successfully met. The need for manual data coding was entirely eliminated, leading to efficient data processing. Furthermore, the strict application of Object-Oriented Programming (OOP) principles resulted in a clean, modular, and easily maintainable codebase. The use of the Tkinter User Interface provided critical real-time, visual feedback, making the application a significantly more engaging educational tool than a traditional Python command-line script.

For future development, potential enhancements include implementing robust error-handling for scenarios like API timeouts or loss of internet connectivity, adding a feature to save the user's highest score locally (e.g., in a `.txt` file), and expanding the GUI to allow users to select their preferred trivia categories and difficulty settings.